Project_Paper_Draft_Group6

Carlos Mabrey, Cody Farris, Olivia Jones, Aaron Eckhardt

2024-12-02

Drum Sample Machine Learning Classifier

Motivation

The motivation behind this project was our love for music and working with audio. For me personally, I produce a lot of music and have my own library of drum sounds that I pulled for this project. I would like to eventually use machine learning techniques on my own music to streamline the production and song-writing process. I have a vision of streaming a track to an AI Agent / database, which segments each part of the song (Vocals, Guitars, Bass, Drums, etc.) and gives suggestions based on the context of the song/database.

Creating a drum classification tool seemed like a good first step to creating an app like this, teaching skills like working with audio data and data manipulation. Working with audio data is not very well doccumented online as it is a niche interest, so this project provides the opportunity to learn skills that interest me!

Data Preperation

Data Collection

The wav files were pulled from a personal library of drum sounds, primarily from Splice. They were automatically filtered based on regex patterns in the file name. The features for each sample were extracted using the CREPE library (dicussed in the next section) manually using local hardware, and were sabed into a CSV titled 'sample_features.csv'. As we found out later, some of the data was messy due to the regex method of creating the dataset, so some data filtering was necessary.

{Plot Distribution of samples}

Data Cleaning

To increase accuracy and omit any outliers, we filtered samples that were longer than 10 seconds (implying the sample may be a loop instead of a one-shot). And further filtered kicks, hats, and snares to be less than 5 seconds. The ride and crash labels were combined into one catecory, We also removed some excess labels, such as perussion, claps, and toms. Out initial generated dataset was very unballanced, shown below.

Initial dataset

kick 471 snare 467 tom 250 clap 138 ride 120 hat 111

Because the accuracies for the different labels varied greatly instrument to instrument, we decided to balance the dataset based on the minimum number of samples in a given category, which was rides. After balancing, the dataset is much smaller, but is also less biased, so we decided to use it for this project.

After filtering + balancing:

hat 103 kick 103 ride 103 snare 103

Feature Engineering

In order to extract features from the dataset, we used a python library called 'crepe', which is a monophonic pitch tracker based on a deep convolutional neural network that operates directly on the time-domain waveform input {https://pypi.org/project/crepe/}.

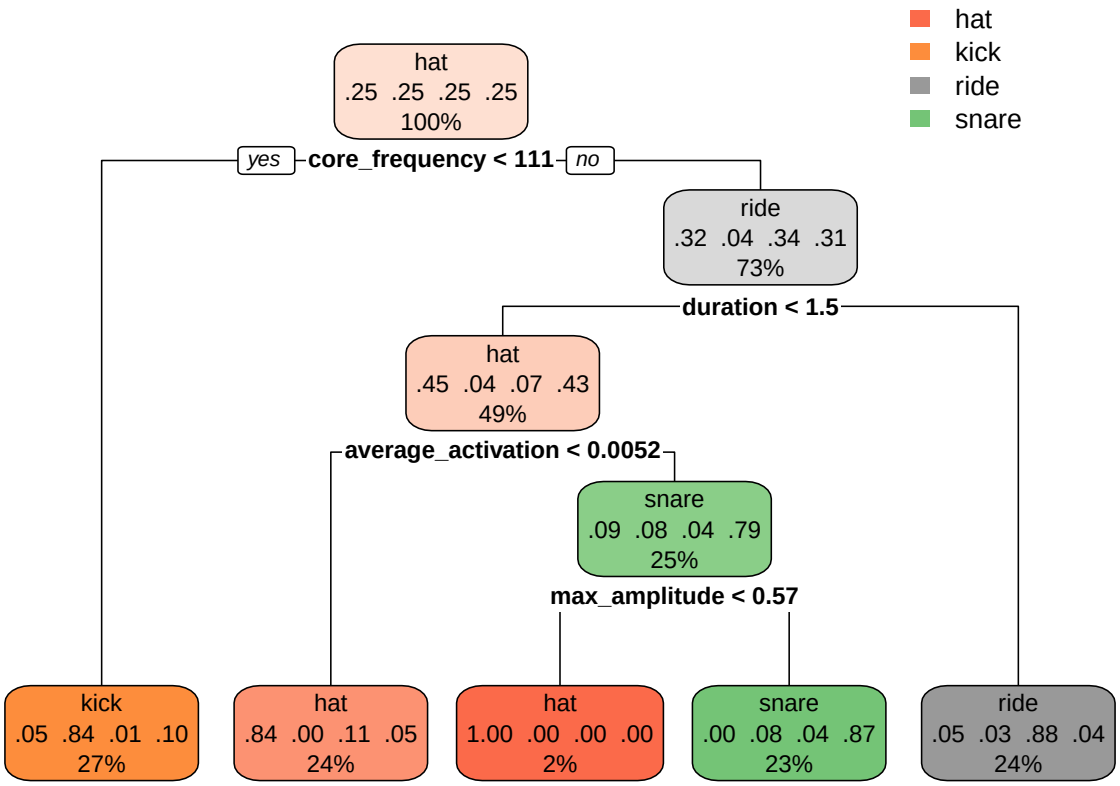
Features: * Core Frequency: Finds frequency thorughout the time-series-sample that has the highest overall confidence {Core Freq Plot} * Max Amplitude: Finds the maximum amplitude (loudness) {Max Amp Plot} * Average Activation: ??? {Activation Plot} * Duration: Length of the sample {Duration Plot}

Methodologies

Tree Model

Initial Tree Model The drums data set is split into 75% training data, and 25% testing data, stratified by the classification 'label'. A basic tree model is fit using the decision_tree() function with five terminal nodes.

The tree has four decisions: - core frequency < 111 - duration < 1.5 - average activation < 0.0052 - max amplitude < 0.57



Predictions and Accuracy The initial model has a testing accuracy of 0.86 and correctly classifies:

- 22/26 Hats

- 25/26 Kicks
- 25/26 Rides
- 18/26 Snares

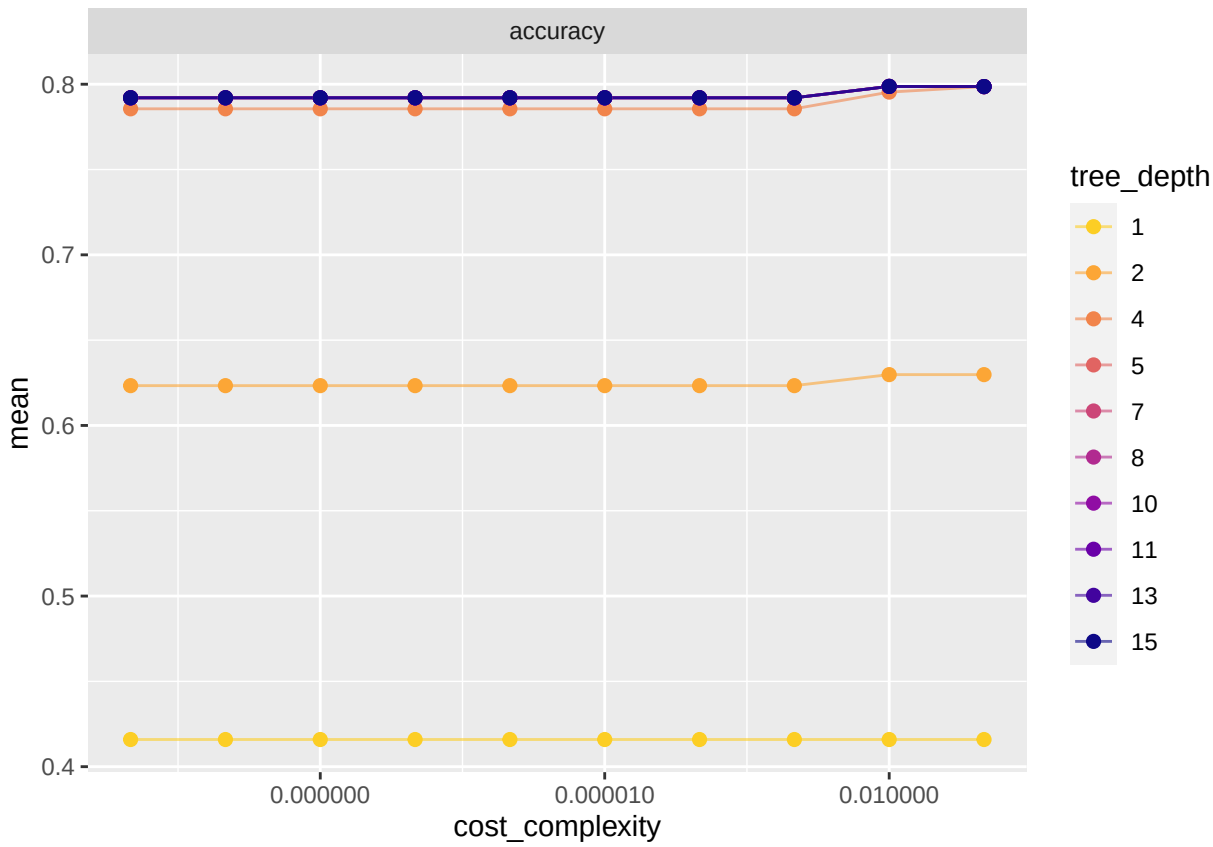
```
##           Reference
## Prediction hat kick ride snare
##      hat    22    3    1     0
##      kick    0   25    1     0
##      ride    1    0   25     0
##      snare   3    4    1    18
```

```
## [1] 0.8653846
```

Tuning Initial Tree Model We used 10-fold cross-validation to find the best cost_complexity parameter and tree depth.

- Best CC Paremeter: 0.01
- Best tree depth: 5

```
## Warning: Ignoring unknown parameters: linewidth
```



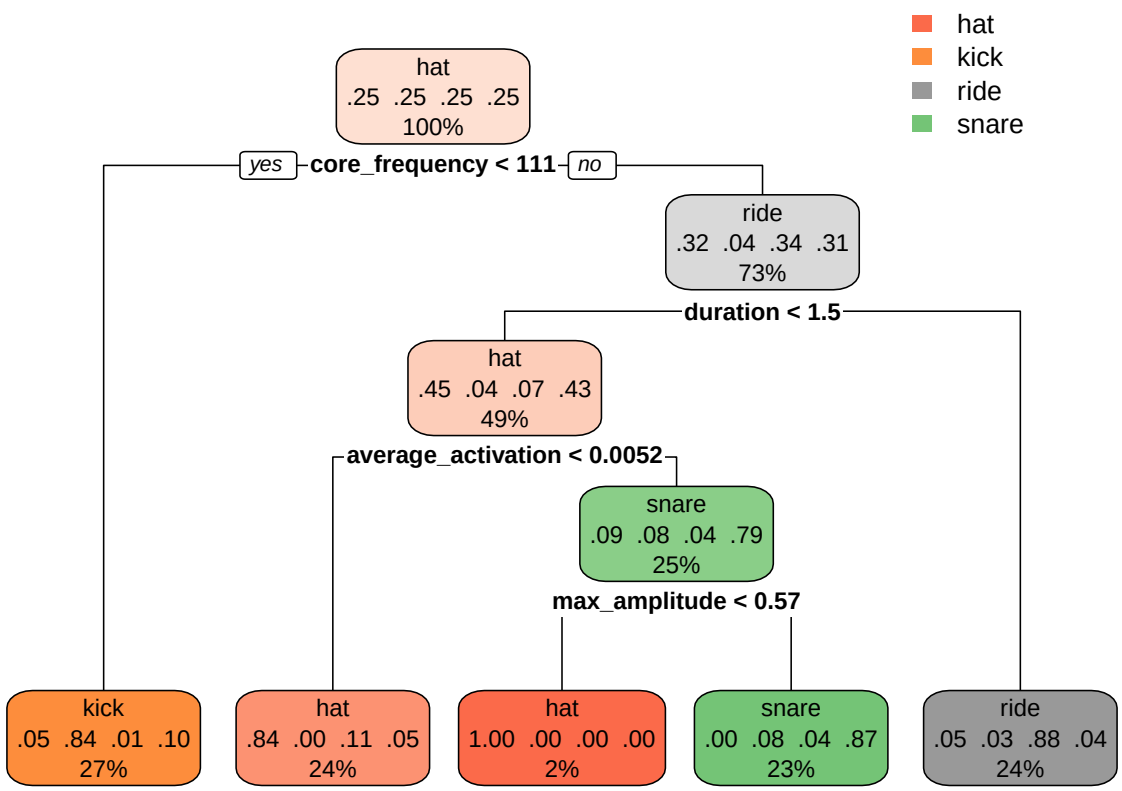

```
## [1] 0.01
```

```
## [1] 5
```

Final Tree Model We refit the tree model using our tuned cost complexity parameter. With the new tuned paramter, we produce the exact same model and predictions as oour initial model.

```
## [1] 0.8653846
```

```
##           Reference
## Prediction hat kick ride snare
##      hat    22    3    1     0
##      kick    0   25    1     0
##      ride    1    0   25     0
##      snare   3    4    1    18
```



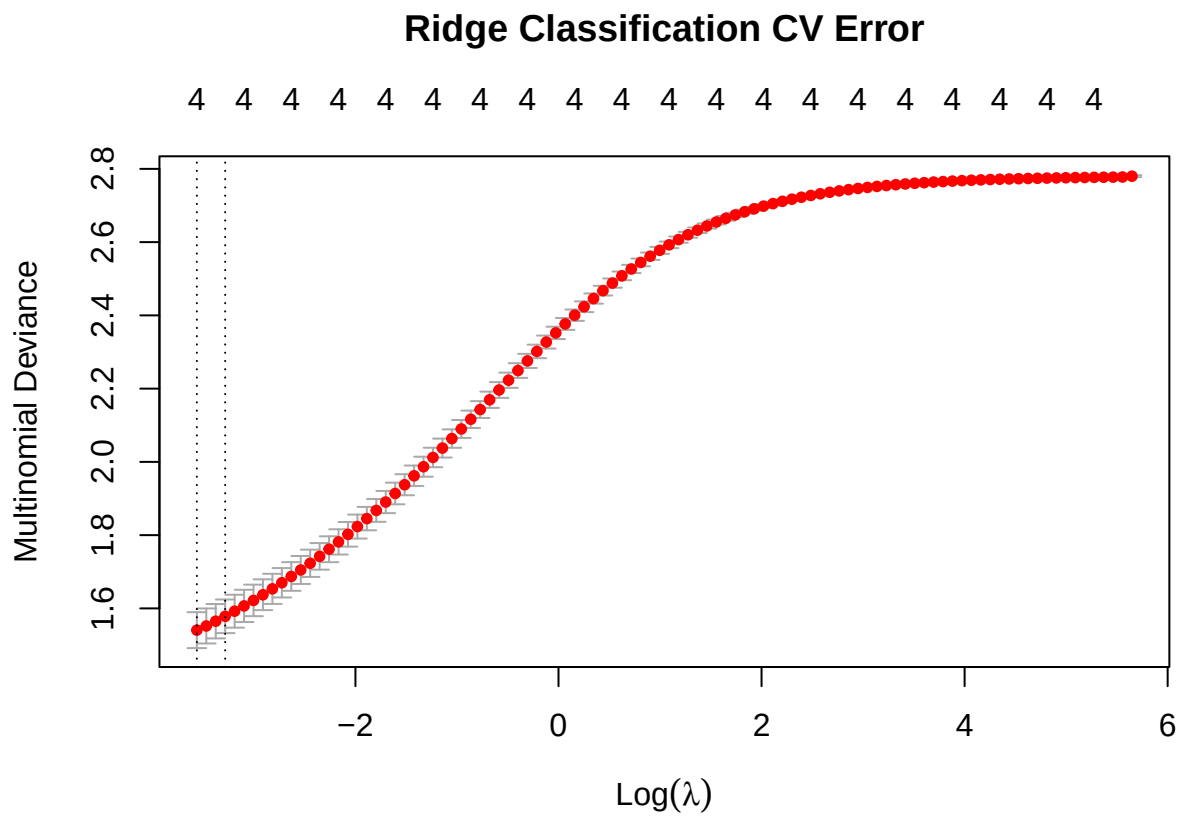
Discussion Although our model did not change at all after tuning the cost complexity parameter, we can explain this with our graph of the 10-fold cross validation step. We can see that changing the tree depth does almost nothing to change our accuracy for all trees with depth higher than 2. The cost complexity parameter has no effect on the accuracy until it has a value of 0.01 or higher, where there is a slight increase in accuracy.

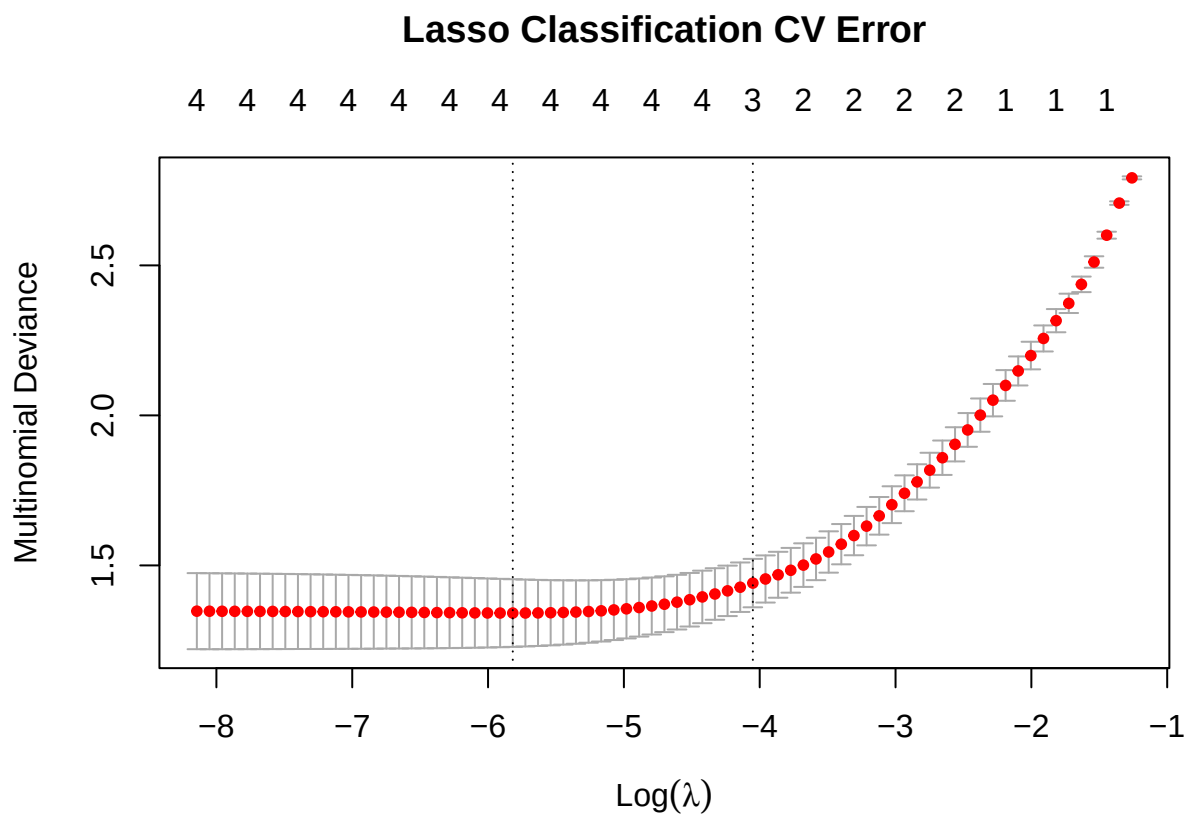
Olivia

```
##   core_frequency max_amplitude average_activation   duration label
## 1      1063.1717  3.059000e+04        0.000640756 0.08730159   hat
## 2       539.3926  1.027420e+09        0.000792523 0.05210884   hat
## 3       540.5202  2.001861e+09        0.002079763 0.10000000   hat
## 4       545.7077  9.874376e+08        0.003245642 0.75523810   hat
## 5      1963.2908  3.643806e-01        0.000855111 0.07061225   hat
## 6       545.5926  1.771449e+09        0.003318707 0.75523810   hat
```

```
##  core_frequency    max_amplitude       average_activation     duration
##  Min.   :  32.39   Min.   :0.000e+00   Min.   :0.0003003   Min.   :0.03231
```

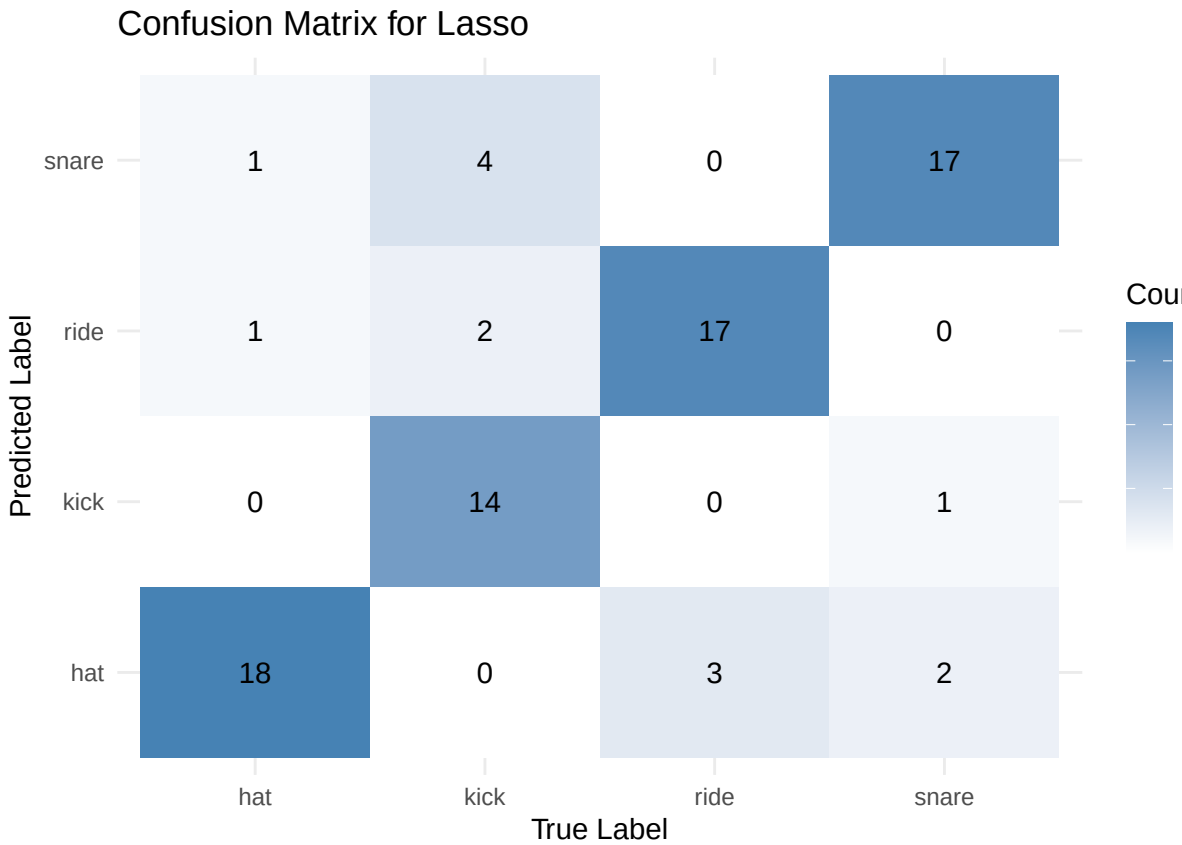
```
##  1st Qu.:  69.50   1st Qu.:2.843e+04   1st Qu.:0.0036056   1st Qu.:0.27357
##  Median : 267.25   Median :1.448e+09   Median :0.0067618   Median :0.67100
##  Mean   : 467.39   Mean   :1.140e+09   Mean   :0.0062646   Mean   :1.47459
##  3rd Qu.: 573.05   3rd Qu.:2.027e+09   3rd Qu.:0.0087507   3rd Qu.:1.81272
##  Max.   :1971.33   Max.   :2.147e+09   Max.   :0.0118515   Max.   :9.60000
##     label
##  Length:412
##  Class :character
##  Mode  :character
##
##
##
```



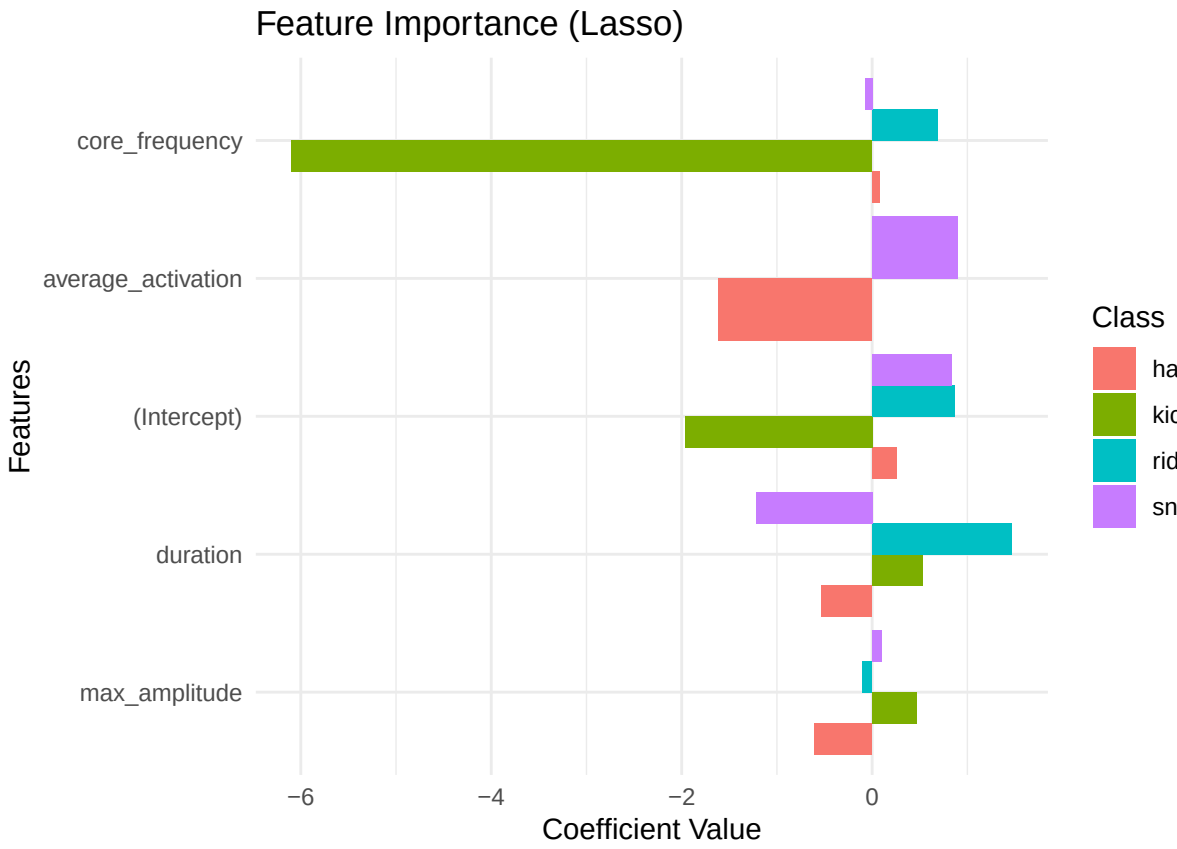



```
## [1] 0.005166839
## [1] 0.02822271
## [1] "Lasso Accuracy: 75.9 %"
## [1] "Ridge Accuracy: 78.31 %"
## [1] "Lasso Accuracy: 82.5 %"
## [1] "Ridge Accuracy: 80 %"
## [1] "Tuned Lasso Accuracy: 82.5 %"
## [1] "Tuned Ridge Accuracy: 80 %"
## [1] "Elastic Net Accuracy: 81.25 %"
## [1] "Lasso Confusion Matrix:"
##                True
## Lasso_Predicted hat kick ride snare
##            hat    18    0    3     2
##            kick    0   14    0     1
##            ride    1    2   17     0
##            snare   1    4    0    17
## [1] "Ridge Confusion Matrix:"
##                True
## Ridge_Predicted hat kick ride snare
##            hat    18    0    3     2
```

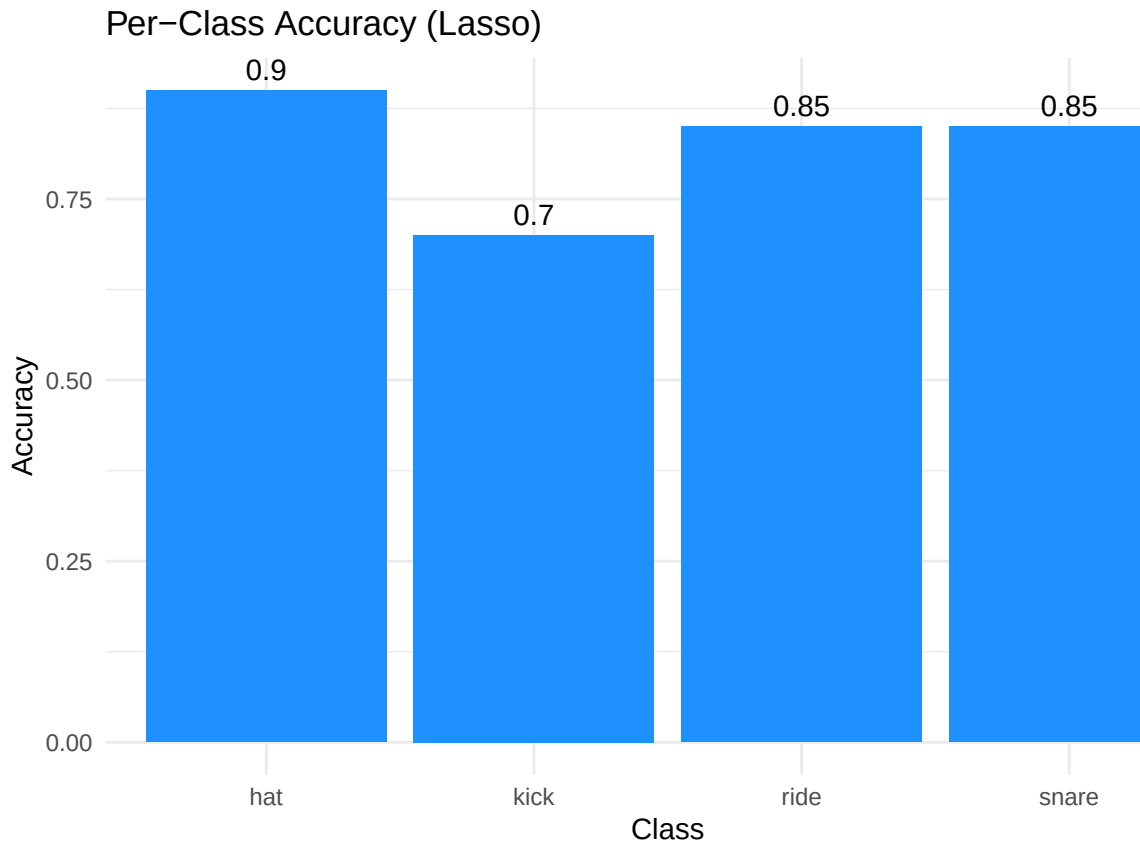
```
##           kick    0   15    1     2
##           ride    1    1   16     1
##           snare   1    4    0    15

## [1] "Elastic Net Confusion Matrix:"

##                  True
## Elastic_Predicted hat kick ride snare
##             hat    18    0    3     2
##             kick    0   14    0     1
##             ride    1    2   17     1
##             snare   1    4    0    16

##
## Attaching package: 'reshape2'

## The following object is masked from 'package:tidyr':
##
##     smiths

##
## Attaching package: 'reshape'

## The following objects are masked from 'package:reshape2':
##
##     colsplit, melt, recast

## The following object is masked from 'package:Matrix':
##
##     expand

## The following object is masked from 'package:dplyr':
##
##     rename

## The following objects are masked from 'package:tidyr':
##
##     expand, smiths

## Warning in type.convert.default(X[[i]], ...): 'as.is' should be specified by the
## caller; using TRUE

## Warning in type.convert.default(X[[i]], ...): 'as.is' should be specified by the
## caller; using TRUE
```



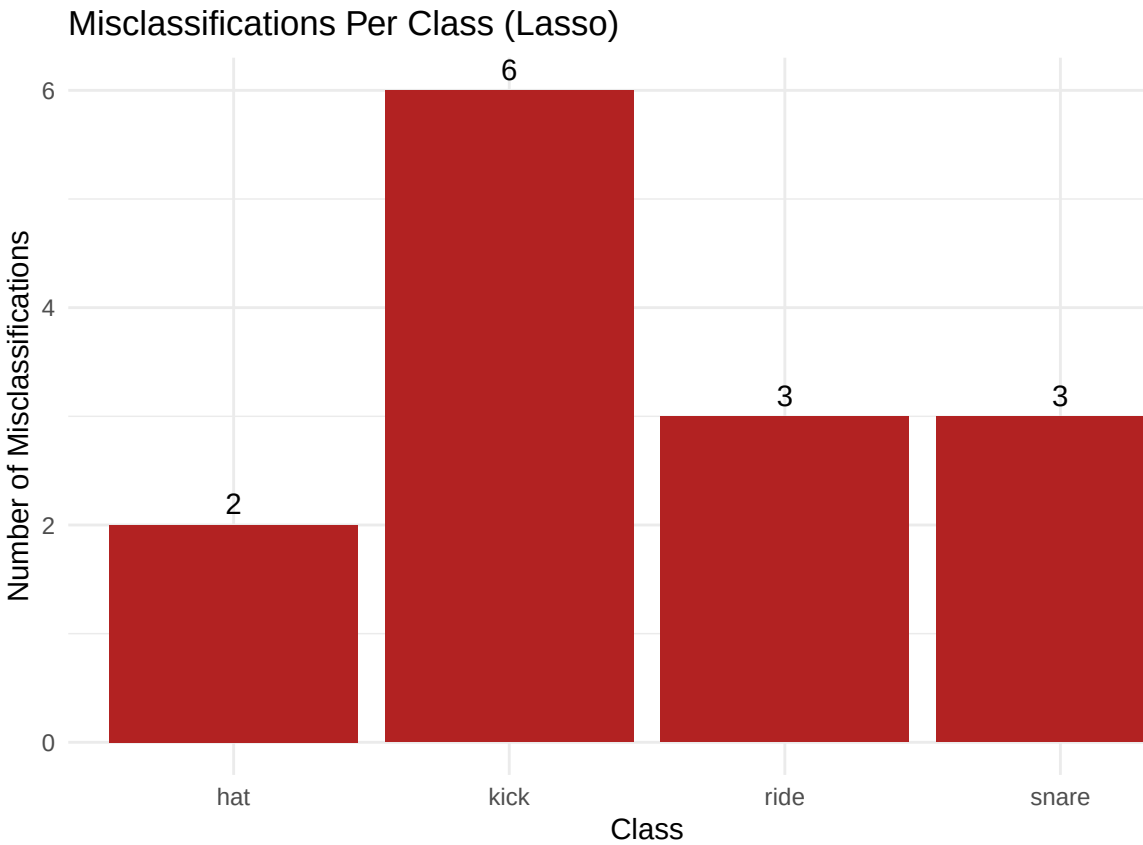
matrix heatmap-1.pdf



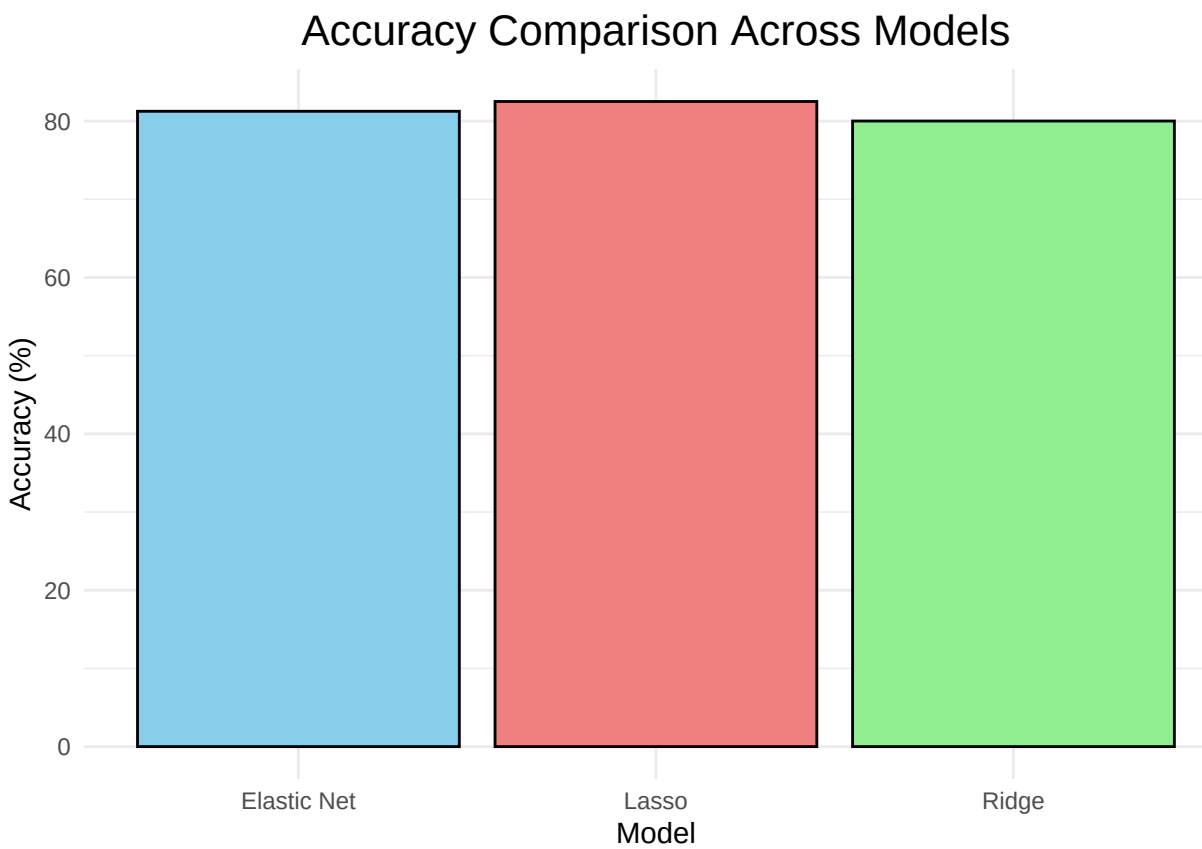
Importance Plot-1.pdf



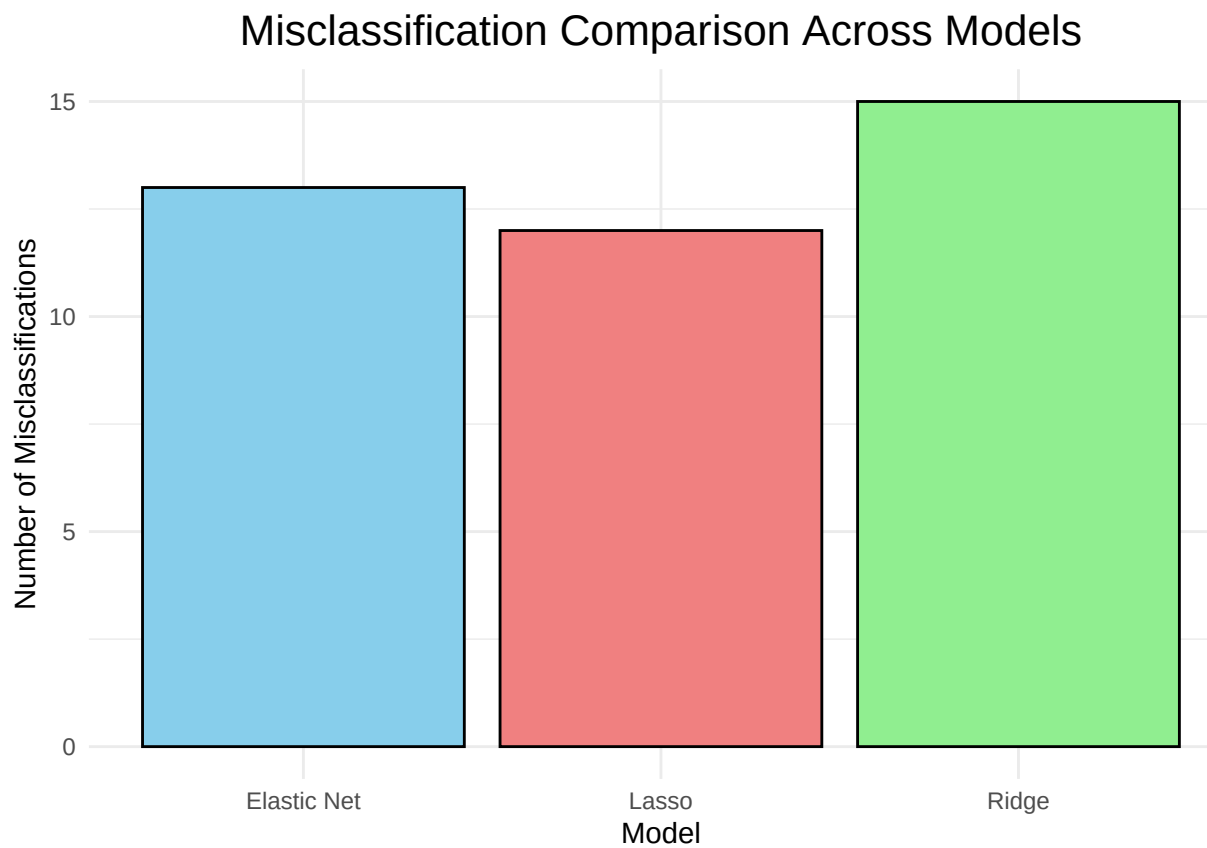
Accuracy Bar Chart-1.pdf



Trends (Bar Chart)-1.pdf



Across Models-1.pdf



Across Models-2.pdf

Aaron

SVM on full dataset Kernel: linear
Best Accuracy: 0.92
Random Seed for Best Iteration: 95405
Confusion Matrix:
[[29 0 0 1]
[1 24 1 1]
[1 0 20 0]
[1 2 0 22]]
Classification Report:
precision recall f1-score support

hat	0.91	0.97	0.94	30
kick	0.92	0.89	0.91	27
ride	0.95	0.95	0.95	21
snare	0.92	0.88	0.90	25
accuracy			0.92	103

macro avg 0.92 0.92 0.92 103 weighted avg 0.92 0.92 0.92 103

Kernel: poly
Best Accuracy: 0.83
Random Seed for Best Iteration: 1032
Confusion Matrix:

[[20 0 2 1]
[0 21 1 5]
[1 0 24 3]
[1 3 0 21]]
Classification Report:
precision recall f1-score support

hat	0.91	0.87	0.89	23
kick	0.88	0.78	0.82	27
ride	0.89	0.86	0.87	28
snare	0.70	0.84	0.76	25
accuracy			0.83	103

macro avg 0.84 0.84 0.84 103 weighted avg 0.84 0.83 0.84 103

Kernel: rbf
Best Accuracy: 0.87
Random Seed for Best Iteration: 1032
Confusion Matrix:
[[23 0 0 0]
[1 22 2 2]
[2 0 25 1]
[3 2 0 20]]
Classification Report:
precision recall f1-score support

hat	0.79	1.00	0.88	23
kick	0.92	0.81	0.86	27
ride	0.93	0.89	0.91	28
snare	0.87	0.80	0.83	25
accuracy			0.87	103

macro avg 0.88 0.88 0.87 103 weighted avg 0.88 0.87 0.87 103

Kernel: sigmoid
Best Accuracy: 0.81
Random Seed for Best Iteration: 77350
Confusion Matrix:
[[18 1 5 0]
[0 25 2 3]
[3 0 22 0]
[1 3 2 18]]
Classification Report:
precision recall f1-score support

hat	0.82	0.75	0.78	24
kick	0.86	0.83	0.85	30
ride	0.71	0.88	0.79	25
snare	0.86	0.75	0.80	24
accuracy			0.81	103

macro avg 0.81 0.80 0.80 103 weighted avg 0.81 0.81 0.81 103

SVM with hat and snare only Kernel: linear
Best Accuracy: 0.96
Random Seed for Best Iteration: 25192
Confusion Matrix:
[[25 1]
[1 25]]
Classification Report:
precision recall f1-score support

```
     hat        0.96       0.96       0.96         26
   snare        0.96       0.96       0.96         26

accuracy                               0.96         52
```

macro avg 0.96 0.96 0.96 52 weighted avg 0.96 0.96 0.96 52

Kernel: poly
Best Accuracy: 0.98
Random Seed for Best Iteration: 11049
Confusion Matrix:
[[26 1] [0 25]] Classification Report:
precision recall f1-score support

```
     hat        1.00       0.96       0.98         27
   snare        0.96       1.00       0.98         25

accuracy                               0.98         52
```

macro avg 0.98 0.98 0.98 52 weighted avg 0.98 0.98 0.98 52

Kernel: rbf
Best Accuracy: 0.96
Random Seed for Best Iteration: 25192
Confusion Matrix:
[[24 2]
[0 26]]
Classification Report:
precision recall f1-score support

```
     hat        1.00       0.92       0.96         26
   snare        0.93       1.00       0.96         26

accuracy                               0.96         52
```

macro avg 0.96 0.96 0.96 52 weighted avg 0.96 0.96 0.96 52

Kernel: sigmoid
Best Accuracy: 0.98
Random Seed for Best Iteration: 25192
Confusion Matrix:
[[26 0]
[1 25]]
Classification Report:
precision recall f1-score support

```
     hat        0.96       1.00       0.98         26
   snare        1.00       0.96       0.98         26

accuracy                               0.98         52
```

macro avg 0.98 0.98 0.98 52 weighted avg 0.98 0.98 0.98 52

SVM without duration Kernel: linear
Best Accuracy: 0.79
Random Seed for Best Iteration: 86565
Confusion Matrix:
[[29 0 1 0]
[1 21 0 0]
[5 0 15 5]
[2 5 3 16]]
Classification Report:
precision recall f1-score support

hat	0.78	0.97	0.87	30
kick	0.81	0.95	0.88	22
ride	0.79	0.60	0.68	25
snare	0.76	0.62	0.68	26
accuracy			0.79	103

macro avg 0.79 0.78 0.78 103 weighted avg 0.78 0.79 0.78 103

Kernel: poly
Best Accuracy: 0.78
Random Seed for Best Iteration: 46480
Confusion Matrix:
[[15 0 1 2]
[1 16 1 7]
[3 1 28 1]
[1 4 1 21]]
Classification Report:
precision recall f1-score support

hat	0.75	0.83	0.79	18
kick	0.76	0.64	0.70	25
ride	0.90	0.85	0.88	33
snare	0.68	0.78	0.72	27
accuracy			0.78	103

macro avg 0.77 0.77 0.77 103 weighted avg 0.78 0.78 0.78 103

Kernel: rbf
Best Accuracy: 0.81
Random Seed for Best Iteration: 40058
Confusion Matrix:
[[29 1 0 0]
[0 24 0 2]
[6 0 13 4]
[1 5 1 17]]
Classification Report:
precision recall f1-score support

hat	0.81	0.97	0.88	30
kick	0.80	0.92	0.86	26
ride	0.93	0.57	0.70	23
snare	0.74	0.71	0.72	24

```
accuracy                           0.81       103
```

macro avg 0.82 0.79 0.79 103 weighted avg 0.82 0.81 0.80 103

Kernel: sigmoid
Best Accuracy: 0.67
Random Seed for Best Iteration: 15892
Confusion Matrix:
[[28 1 5 1]
[0 22 0 3]
[5 3 7 8]
[1 5 2 12]]
Classification Report:
precision recall f1-score support

```
     hat       0.82      0.80      0.81        35
    kick       0.71      0.88      0.79        25
    ride       0.50      0.30      0.38        23
   snare       0.50      0.60      0.55        20

accuracy                           0.67       103
```

macro avg 0.63 0.65 0.63 103 weighted avg 0.66 0.67 0.66 103

Issues

The first issue we ran into was feature selection. The issues we ran into were mainly low accuracies from the original dataset.

Improvements

Through data cleaning techniques, we were able to greatly improve the accuracy of the model. Initially, we were getting accuracies of around 60%, which we were able to improve to 88% with a Random Forrest Model.

{Include Tables comparing the initial results before the dataset was cleaned}

Model after improvements {Include Tables comparing the initial results after the dataset was cleaned}

Further Analysis PCA Visualization of feature space to explain support vector

Conclusion References